1. RMI / RPC

Rmi

```
import csv

class TrustMeshServer:

 def processData (self,param):
 print(f"[Server] Process Parameter --> {param}")  return
param + 5



def invoke(server,methodName,param):
 method = getattr(server,methodName)

 return method(param)

with open('rpc.csv','r') as file:
 data = list(csv.DictReader(file))

server = TrustMeshServer()

for row in data:
 callID = row['call_id']
 client = row['client_name']
 methodName = row['method_name']
 param = row['param_value']
 ts = row['timestamp']

 print(f"\n[Invoke] {client} invoking {methodName}({param})") r =

invoke(server,methodName,int(param))

 print(f"[Return] Result --> {r}")
```

Rpc

```
import csv

def processData (param):
 print(f"[Server] Processing Parameter --> {param}")  return
param + 5

def dispatcher (methodName , param):
 if methodName == "processData":
 return processData(int(param))

 else:
 return "Unknown Method NAme"

with open('rpc.csv','r') as file:
 data = list(csv.DictReader(file))

for row in data:
 callID = row['call_id']
 client = row['client_name']
 methodName = row['method_name']
 param = row['param_value']
 ts = row['timestamp']

 print(f"\n[Dispatch] {ts} : Client Inovking Method : {client} -->
{methodName}({param})")

 r = dispatcher(methodName,param)

 print(f"[Return] Result --> {r}")
```

Csv

```
call_id,client_name,method_name,param_value,timestamp
101,Client_A,processData,450,10:00:01
102,Client_B,processData,720,10:00:05
103,Client_C,processData,350,10:00:09
104,Client_A,processData,900,10:00:15
105,Client_B,processData,550,10:00:20
```

2. Multithreading

```
import csv
import threading
import time

with open('lock.csv','r') as f:
 data = list(csv.DictReader(f))

lock = threading.Lock()
completed = []

def processTask (task):
 tid = task['task_id']
 threadName = task['thread_name']
 burst = int(task['burst_ms'])
 priority = task['priority']

 print(f"[THREAD] {threadName} started task {tid}")
print(f"[WAITING] {threadName} waiting for lock -> {tid}")

 with lock:

 print(f"[Lock Acquired] {threadName} --> {tid}")  print(
 f"[PROCESSING] {threadName} processing "  f"{tid} |
Priority: {priority}"
 )

 time.sleep(burst/1000)

 completed.append(task)

 print(f"[COMPLETED] {tid}")

 print(f"[LOCK RELEASED] {threadName} -> {tid}") thread = []

for task in data:
 t = threading.Thread(
 target=processTask,
 args=(task,)
```

```
 )

 thread.append(t)
 t.start()

for t in thread:
 t.join()

print("\n===== FINAL COMPLETED TASKS =====")

for task_id in completed:
 print(task_id)
```

Csv:

task_id,thread_name,burst_ms,priority,status
T1,Worker_1,200,HIGH,PENDING
T2,Worker_2,300,MEDIUM,PENDING
T3,Worker_3,150,HIGH,PENDING
T4,Worker_1,250,LOW,PENDING
T5,Worker_2,100,HIGH,PENDING
T6,Worker_3,200,MEDIUM,PENDING

3. Lamport

```
import csv

with open('lamport.csv','r') as f:
 data = list(csv.DictReader(f))

clocks = {}
messages = {}

events = []

for d in data:
 eid = d['event_id']
 pid = d['process_id']
 type = d['event_type']
```

```
 msgTo = d['message_to']
 initClock = int(d['initial_clock'])

 if pid not in clocks:
 clocks[pid] = initClock

 old = clocks[pid]

 if type == "LOCAL":
 clocks[pid] += 1
 print("\n"
 f"{eid} | {pid} | Local |"  f" L: {old} -->
{clocks[pid]}"  )

 elif type == "SEND":
 clocks[pid] += 1
 messages[msgTo] = clocks[pid]

 print("\n"
 f"{eid} | {pid} | Send --> {msgTo} |"  f" L: {old} -->
{clocks[pid]}"  )

 print(
 f"Message TS : {clocks[pid]}"
 )

 else:
 t = messages.get(pid,0)
 clocks[pid] = max (
 clocks[pid],t
 ) + 1

 print("\n"
 f"{eid} | {pid} | RECEIVE | "
 f"L: max({old}, {t}) + 1 "
 f"= {clocks[pid]}"
 )
```

```
 events.append(
 {
 'event_id' : eid,
 'process_id' : pid,
 'event_type' : type,
 'clock' : clocks[pid]
 }
 )

events.sort(key = lambda x : (x['clock'],x['process_id']))

print ("\nEVents Order")
i = 1
for event in events:
 print(
 i,
 event["event_id"],
 event["process_id"],
 event["event_type"],
 "Clock =", event["clock"]
 )
 i += 1

print("\nClocks of all")
for i in clocks:
 print(
 f"{i} = {clocks[i]}"
 )
```

CSV

event_id,process_id,event_type,message_to,initial_clock
E1,P1,LOCAL,,1
E2,P1,SEND,P2,2
E3,P2,RECEIVE,,0
E4,P2,LOCAL,,1
E5,P2,SEND,P3,2
E6,P3,RECEIVE,,0
E7,P1,LOCAL,,3
E8,P3,LOCAL,,1

4. Cristian's Algorithm

```
import csv

with open('christian.csv','r') as f:
 data = list(csv.DictReader(f))

for d in data:
 cid = d['client_id']
 t0 = int(d['t0_send_ms'])
 tS = int(d['t_server_ms'])
 t1 = int(d['t1_recv_ms'])

 rtt = t1 - t0
 estTime = tS + (rtt/2)
 offset = estTime - t1

 print(f"\n{cid}")
 print(f"t0: {t0} | t1: {t1} | tS: {tS}")
 print(f"RTT = {t1} - {t0} = {rtt} ms")
 print(f"Estimated Time = {tS} + ({rtt}/2) = {estTime} ms")
print(f"Offset = {estTime} - {t1} = {offset} ms")
```

client_id,t0_send_ms,t_server_ms,t1_recv_ms

C4,1021,1099,1119
C5,1459,1526,1540
C3,1474,1570,1605
C5,1747,1825,1846
C5,1955,2047,2074
C2,2184,2242,2238
C3,2215,2291,2316
C3,3300,3378,3401
C3,3324,3400,3410
C5,3433,3509,3534
C4,3853,3932,3943
C3,3904,4010,4051
C5,4385,4466,4479
C2,4485,4565,4588

5. Berkeley

```
import csv

with open('berkeley.csv','r') as f:
 data = list(csv.DictReader(f))

faulty_nodes = []

sum = 0

for d in data:
 if d['is_faulty'] == "TRUE":
 faulty_nodes.append(d)

 else:
 sum += int(d['clock_time_sec'])

print(
 f"Faulty nodes:", [i["node_id"] for i in faulty_nodes] )

nodes = int(len(data)) - int(len(faulty_nodes))
avg = sum/nodes

print(
 f"\n[Master] Avg Cal | Sum = {sum} | Nodes = {nodes}"
f"\n[Master] Avg = {sum}/{nodes} = {avg: .3f}"
)

for d in data:
 if d['is_faulty'] == "FALSE":
 print(f"\nNode {d['node_id']}")
 offset = avg - int(d['clock_time_sec'])
 print(f"Node Time = {d['clock_time_sec']} || Master = {avg: .3f}")
print(f"Offset = {avg: .3f} - {d['clock_time_sec']} = {offset: .3f}")
```

Csv

node_id,clock_time_sec,is_faulty
N1,50412,FALSE
N2,50395,FALSE
N3,50405,FALSE
N4,42100,TRUE
N5,50388,FALSE
N6,50418,FALSE
N7,59250,TRUE
N8,50402,FALSE

6. Vector

```
import csv

with open('vector.csv','r') as f:
 data = list(csv.DictReader(f))

clocks = {
 'P1' : {
 'P1' : 0,
 'P2' : 0,
 'P3' : 0
 },
 'P2' : {
 'P1' : 0,
 'P2' : 0,
 'P3' : 0
 },
 'P3' : {
 'P1' : 0,
 'P2' : 0,
 'P3' : 0
 }
}

messages = {}
events = {}
```

```
#event_id,process_id,event_action,target_process

for e in data:
 eid = e['event_id']
 pid = e['process_id']
 type = e['event_action']
 msgTo = e['target_process']

 old = clocks[pid].copy()

 if type == "LOCAL":
 clocks[pid][pid] += 1
 print(
 f"\n{eid} | {pid} | LOCAL |"  f"\nCLock: {old} -->
{clocks[pid]}"  )

 elif type == "SEND":
 clocks[pid][pid] += 1
 messages[msgTo] = clocks[pid].copy()  print(
 f"\n{eid} | {pid} | SEND --> {msgTo} |"  f"\nCLock:
{old} --> {clocks[pid]}"  )
 print(
 f"Message TS : {messages[msgTo]}"  )

 else:
 t = messages.get(pid)

 for p in clocks:
 clocks[pid][p] = max(
 t[p],clocks[pid][p]
 )

 clocks[pid][pid] += 1

 print(
  f"\n{eid}  |  {pid}  |  Recieve  |"    f"\nCLock:
max({old},{t}) + 1"  f"\nClock: {clocks[pid]}"
```

```
 )
 events[eid] = {
 'process_id': pid,
 'event_type': type,
 'clock': clocks[pid].copy()
 }

print("\nEvents")
for e in events:
 print(
 f"{e} --> {events[e]['clock']}"
 )

print("\nProcess Wise Clocks")
for process in clocks:
 print(process, "=", clocks[process])
```

Csv

event_id,process_id,event_action,target_process
E1,P1,LOCAL,
E2,P1,SEND,P2
E3,P2,RECEIVE,
E4,P2,LOCAL,
E5,P2,SEND,P3
E6,P3,RECEIVE,
E7,P1,LOCAL,
E8,P3,SEND,P1
E9,P1,RECEIVE,

7. Bully Algo

```
import csv
import random

with open('bully.csv','r') as f:
 data = list(csv.DictReader(f))

curr_cord = []
alive_nodes = []
#node_id,is_alive,is_current_coordinator
for d in data:
 if d['is_alive'] == "FALSE" and d['is_current_coordinator'] == "TRUE":
curr_cord.append(d['node_id'])
 elif d['is_alive'] == "TRUE":
 alive_nodes.append(d['node_id'])

idx = random.randint(0,len(alive_nodes)-1)
initiator = alive_nodes[idx]

print(
 f"Node {initiator} ---> Leader Node {curr_cord[0]}"
 f"\nNo Response Timeout | Leader is Dead"
)

print(
 f"\nNode {initiator} Starts the Election !"
)

candidates = []

for d in data :
 if d['is_alive'] == "TRUE" and d['node_id'] > initiator:
candidates.append(d['node_id'])
 print(
 f"\n[Election Msg] Node {initiator} --> Node {d['node_id']}"  f"\n[Reply]
Response OK"
 )
 elif d['is_alive'] == "FALSE" and d['node_id'] > initiator:  print(
```

```
 f"\n[Election Msg] Node {initiator} --> Node {d['node_id']}"
 f"\n[Reply] No Response Timeout"
 )

for c in candidates.copy():
 print(
 f"\nNode {c} Starts the Election !"
 )
 for d in data :
 if d['is_alive'] == "TRUE" and d['node_id'] > c:  print(
 f"\n[Election Msg] Node {c} --> Node {d['node_id']}"  f"\n[Reply]
Response OK"
 )
 elif d['is_alive'] == "FALSE" and d['node_id'] > c:  print(
 f"\n[Election Msg] Node {c} --> Node {d['node_id']}"  f"\n[Reply] No
Response Timeout"
 )

leader = max(alive_nodes)

print(f"\n[COORDINATOR] Node {leader} becomes the new Leader")

for node in alive_nodes:
 if node != leader:
 print(
 f"[BROADCAST] Leader Node {leader} --> Node {node}"  )
```

Csv

node_id,is_alive,is_current_coordinator
1,TRUE,FALSE
2,TRUE,FALSE
3,TRUE,FALSE
4,FALSE,TRUE
5,TRUE,FALSE
6,TRUE,FALSE

8. Ring Algo

```
import csv
import random

with open('ring.csv','r') as f:
 data = list(csv.DictReader(f))

#node_id,next_node_id,is_active,priority_rank dead
= []
alive = []

for d in data:
 if d['is_active'] == "FALSE":
 dead.append(d['node_id'])
 else:
 alive.append(d['node_id'])

idx = random.randint(0,len(alive)-1)
initiator = alive[idx]

print(
 f"Node {initiator} ---> Leader Node {dead[0]}"  f"\nNo
Response Timeout | Leader is Dead" )

print(
 f"\nNode {initiator} Starts the Election !" )

candidates = []

curr = initiator

while True:
 for d in data:
 if d['node_id'] == curr:
 row = d
 break

 if row['is_active'] == "TRUE":
 print(
```

```
 f"\n[ELecttion Token] Election Token at Node {curr}"  )
 candidates.append((row['node_id'],row['priority_rank']))  curr =
row['next_node_id']

 if row['is_active'] == "FALSE":
 curr = row['next_node_id']

 if curr == initiator:
 break

print("\nCandidates:", candidates)

leader = max(candidates, key=lambda x: (int(x[1]), int(x[0])))

print(f"\n[COORDINATOR] Node {leader[0]} becomes the new Leader")
```

Csv

node_id,next_node_id,is_active,priority_rank
1,2,TRUE,85
2,3,TRUE,72
3,4,TRUE,91
4,5,FALSE,95
5,6,TRUE,80
6,1,TRUE,88

9. Load Balancer

```
import csv

with open('loadBal.csv','r') as f:
 data = list(csv.DictReader(f))

worker = [['W1',0], ['W2',0], ['W3',0]] curr
= 0

#job_id,compute_units,assigned_worker,status

for d in data:
```

```
 jid = d['job_id']
 units = int(d['compute_units'])
 worker[curr][1] += units

 print(
  f"\nJob  {jid}  -->  Worker  {worker[curr][0]}"    f"  |
{worker[curr][0]} : {worker[curr][1]}"  )

 curr = (curr+1) % len(worker)

print(worker)

worker = [['W1',0], ['W2',0], ['W3',0]] for d
in data:
 jid = d['job_id']
 units = int(d['compute_units'])
 curr = min(worker, key = lambda x : x[1])  curr[1]

+= units

 print(
 f"\nJob {jid} --> Worker {curr[0]}"  f" |
{curr[0]} : {curr[1]}"
 )

print(worker)
```

Csv

job_id,compute_units,assigned_worker,status
J101,35,UNASSIGNED,PENDING
J102,20,UNASSIGNED,PENDING
J103,45,UNASSIGNED,PENDING
J104,15,UNASSIGNED,PENDING
J105,30,UNASSIGNED,PENDING
J106,25,UNASSIGNED,PENDING
J107,50,UNASSIGNED,PENDING
J108,10,UNASSIGNED,PENDING

J109,40,UNASSIGNED,PENDING
J110,20,UNASSIGNED,PENDING
J111,35,UNASSIGNED,PENDING
J112,15,UNASSIGNED,PENDING

10. MPI

```
import csv
from multiprocessing import Pool

with open('matrix.csv','r') as f:
 data = list(csv.DictReader(f))

A = []

for row in data:
 A.append([
 int(row['col_0']),
 int(row['col_1']),
 int(row['col_2']),
 int(row['col_3'])
 ])

B = [
 [1,2],
 [2,3],
 [1,4],
 [3,1]
]

def multiply (task):
 worker, row_id, row = task

 print(f"[WORKER {worker}] Received row {row_id}")  result

= []
```

```
 for i in range(2):
 total = 0
 for j in range(4):
 total += row[j] * B[j][i]
 result.append(total)

 print(f"[WORKER {worker}] {row_id} -> {result}")
 return result
if __name__ == "__main__":

 tasks = []
 print("\n[MASTER] Scattering rows...")

 for i in range(len(A)):
 worker = (i%3) + 1
 print(f"[MASTER] Row {data[i]['row_id']} --> Worker {worker}")

tasks.append([worker,data[i]['row_id'],A[i]])

 with Pool(3) as p:
 r = p.map(multiply,tasks)

 print("\n[MASTER] Gathering results...")

 print("\n===== FINAL MATRIX =====")
 for row in r:
 print(row)
```

Csv

row_id,col_0,col_1,col_2,col_3
R0,2,4,1,5
R1,3,1,2,4
R2,5,2,3,1
R3,1,3,4,2